

M08A Final Project: Library Management System

Project Title: Library Management System

Project Overview

This project simulates a small **Library Management System** that manages books, patrons, and transactions (borrowing and returning). It demonstrates **OOP design** and **advanced C++ features** in an integrated, real-world context.

This **C++ project and design** requires: **four or more classes**, **OOP principles** (inheritance, polymorphism, encapsulation), **operator overloading**, **exception handling**, **collections**, **file I/O**, and **enumerations**.

Core Object-Oriented Requirements

Concept	Implementation
Classes (4+)	Book , Patron , Transaction , Library , and optionally Date
Inheritance	EBook and PrintedBook inherit from Book
Polymorphism	Book has virtual methods (e.g., displayInfo()) overridden by child classes
Operator Overloading	Overload == and << for Book and Patron
Exception Handling	Handle invalid input, missing files, and invalid transactions
Collections	Use std::vector , std::map for storing books and patrons
File I/O	Load and save book and patron data from .txt or .csv files
Enumerations	Define Genre and BookStatus enums
Other Constructs	const correctness, dynamic memory, iterators, and function templates (optional)

M08A Final Project: Library Management System

Additional Required Features

- Implement a **menu-driven interface** for user interaction.
- Add **search** by author or genre.
- Use **templates** for file save/load operations.
- Add a **date/time class** for due dates.

Class Design

1. Base Class: Book

```
enum class Genre { Fiction, NonFiction, Mystery, Science, Biography };  
enum class BookStatus { Available, CheckedOut };
```

```
class Book {  
protected:  
    std::string title;  
    std::string author;  
    Genre genre;  
    BookStatus status;  
public:  
    Book(std::string t, std::string a, Genre g);  
    virtual ~Book() = default;  
  
    virtual void displayInfo() const;  
    void setStatus(BookStatus s);  
    BookStatus getStatus() const;  
  
    bool operator==(const Book& other) const;  
    friend std::ostream& operator<<(std::ostream& os, const Book& b);  
};
```

2. Derived Classes: EBook and PrintedBook

```
class EBook : public Book {  
    double fileSizeMB;  
public:  
    EBook(std::string t, std::string a, Genre g, double size);  
    void displayInfo() const override;  
};
```

```
class PrintedBook : public Book {
```

M08A Final Project: Library Management System

```
    int pageCount;
public:
    PrintedBook(std::string t, std::string a, Genre g, int pages);
    void displayInfo() const override;
};
```

3. Base Class: Patron

```
class Patron {
    std::string name;
    int id;
    std::vector<Book*> borrowedBooks;
public:
    Patron(std::string n, int i);
    void borrowBook(Book* b);
    void returnBook(Book* b);
    void displayPatron() const;

    bool operator==(const Patron& other) const;
    friend std::ostream& operator<<(std::ostream& os, const Patron& p);
};
```

4. Base Class: Transaction

```
#include <ctime>

class Transaction {
    int patronId;
    std::string bookTitle;
    std::string date;
public:
    Transaction(int pid, std::string bt);
    void displayTransaction() const;
};
```

5. Base Class: Library

```
class Library {
    std::vector<Book*> books;
    std::vector<Patron> patrons;
    std::vector<Transaction> transactions;
```

M08A Final Project: Library Management System

```
public:
    ~Library();
    void loadData();
    void saveData();
    void addBook(Book* b);
    void addPatron(const Patron& p);
    void checkoutBook(int patronId, std::string title);
    void returnBook(int patronId, std::string title);
    void displayBooks() const;
    void displayPatrons() const;
};
```

Example Functional Flow

1. The project should load books and patrons from text (csv) files.
2. User can:
 - View all books and patrons
 - Check out or return books of a valid patron
 - Add new books or patrons
3. All transactions are logged in a file.
4. Errors such as missing books, invalid IDs, or unavailable books trigger **exceptions**.

Example File I/O

Books.txt (you are required to add additional books)

Fiction,The Great Gatsby,F. Scott Fitzgerald,Printed,180
Science,Quantum Physics 101,Albert Einstein,EBook,4.5

Patrons.txt (you are required to add additional patrons)

1,Jane Doe
2,John Smith

M08A Final Project: Library Management System

File Operations Example

```
void Library::loadData() {  
    std::ifstream file("books.txt");  
    if (!file) throw std::runtime_error("Failed to open books file.");  
    // parse and create Book objects  
}
```

Example Exception Handling

```
try {  
    library.checkoutBook(1, "The Great Gatsby");  
} catch (const std::exception& e) {  
    std::cerr << "Error: " << e.what() << std::endl;  
}
```

Example Main Function

```
int main() {  
    Library library;  
    try {  
        library.loadData();  
        library.displayBooks();  
        library.displayPatrons();  
  
        library.checkoutBook(1, "The Great Gatsby");  
        library.returnBook(1, "The Great Gatsby");  
        library.saveData();  
    }  
    catch (const std::exception& e) {  
        std::cerr << "Application Error: " << e.what() << std::endl;  
    }  
    return 0;  
}
```